

Paso a paso corregido de los archivos JavaScript

Este documento se centra en funciones.js y script.js. Esta version corrige las notas anteriores: los textos con tildes no se tratan como error y el codigo revisado ya tiene varias correcciones aplicadas.

Parte A: funciones.js

Paso 1: Exportar ConversorTemperatura

La palabra export permite que script.js pueda importar esta clase. Como se usa export/import, el HTML debe cargar script.js con type="module".

```
export class ConversorTemperatura {
```

Paso 2: Inicializar valores

El constructor prepara tres propiedades en cero. Son los espacios donde el objeto guarda la temperatura actual.

```
  constructor() {  
    this.celsius = 0;  
    this.fahrenheit = 0;  
    this.kelvin = 0;  
  }
```

- this.celsius guarda el valor en Celsius.
- this.fahrenheit guarda el valor en Fahrenheit.
- this.kelvin guarda el valor en Kelvin.

Paso 3: Metodo celsiusToFahrenheit

Lee this.celsius, aplica la formula, guarda el resultado en this.fahrenheit y lo devuelve.

```
  celsiusToFahrenheit() {  
    this.fahrenheit = (this.celsius * 9 / 5) + 32;  
    return this.fahrenheit;  
  }
```

Paso 4: Metodo celsiusToKelvin

Lee this.celsius, suma 273.15, guarda el resultado en this.kelvin y lo devuelve.

```
  celsiusToKelvin() {  
    this.kelvin = this.celsius + 273.15;  
    return this.kelvin;  
  }
```

Paso 5: Metodos desde Fahrenheit

Cuando el usuario elige Fahrenheit, script.js llena conversor.fahrenheit. Luego estos metodos calculan Celsius y Kelvin.

```
  fahrenheitToCelsius() {  
    this.celsius = (this.fahrenheit - 32) * 5 / 9;  
    return this.celsius;  
  }
```

```
fahrenheitToKelvin() {
  this.kelvin = (this.fahrenheit - 32) * 5 / 9 + 273.15;
  return this.kelvin;
}
```

Paso 6: Metodos desde Kelvin

Cuando el usuario elige Kelvin, script.js llena conversor.kelvin. Luego estos metodos calculan Celsius y Fahrenheit.

```
kelvinToCelsius() {
  this.celsius = this.kelvin - 273.15;
  return this.celsius;
}

kelvinToFahrenheit() {
  this.fahrenheit = (this.kelvin - 273.15) * 9 / 5 + 32;
  return this.fahrenheit;
}
```

Paso 7: Clase validador

La clase validador permite validar un input por separado. En el flujo actual no se importa ni se instancia, asi que no participa en la conversion.

- Puede quedarse en funciones.js sin afectar la app.
- Si luego decides usarla, los IDs que reciba deben existir en index.html.

Parte B: script.js

Paso 8: Importar la clase

script.js trae ConversorTemperatura desde funciones.js. Esto permite usar los metodos de conversion.

```
import { ConversorTemperatura } from './funciones.js';
```

Paso 9: Crear una instancia

new ConversorTemperatura() ejecuta el constructor y crea un objeto listo para guardar temperaturas.

```
const conversor = new ConversorTemperatura();
```

Paso 10: Conectar con el HTML

document.getElementById busca cada elemento por su id. Esta parte ya coincide con tu HTML actual.

```
const inputTemperatura = document.getElementById('temperatura');
const selectUnidad = document.getElementById('unidad-medida');
const mensaje = document.getElementById('mensaje');
const resultado = document.getElementById('resultado');
const botonConvertir = document.getElementById('convertir');
```

Paso 11: Esperar el click

El programa no convierte apenas carga la pagina. Espera a que el usuario presione el

boton Convertir.

```
botonConvertir.addEventListener('click', () => {  
    // codigo que se ejecuta al hacer click  
});
```

Paso 12: Leer valor y unidad

value toma el contenido actual. trim() elimina espacios al inicio o al final del input.

```
const valor = inputTemperatura.value.trim();  
const unidad = selectUnidad.value;
```

Paso 13: Validar si esta vacio

Si valor esta vacio, se muestra un mensaje, se limpia el resultado y return detiene la funcion.

```
if (valor === '') {  
    mensaje.textContent = 'El campo no puede estar vacio.';  
    mensaje.style.color = 'red';  
    resultado.textContent = '';  
    return;  
}
```

Nota: en tu archivo el texto puede tener tilde. Eso esta bien; aqui se muestra sin tilde solo por compatibilidad del generador simple de PDF.

Paso 14: Convertir texto a numero

Aunque el input sea type="number", JavaScript lee su valor como texto. Por eso se usa Number(valor).

```
const temperatura = Number(valor);
```

Paso 15: Validar numero

Number.isFinite confirma que el valor convertido sea un numero valido. Si no lo es, se detiene.

```
if (!Number.isFinite(temperatura)) {  
    mensaje.textContent = 'Por favor, ingresa un numero valido.';  
    mensaje.style.color = 'red';  
    resultado.textContent = '';  
    return;  
}
```

Paso 16: Mostrar validacion positiva

Si pasa las validaciones, el mensaje cambia a verde.

```
mensaje.textContent = 'Valor valido.';  
mensaje.style.color = 'green';
```

Paso 17: Convertir desde Celsius

Si unidad es celsius, se guarda la temperatura en conversor.celsius y se calculan Fahrenheit y Kelvin.

```
if (unidad === 'celsius') {
```

```

    conversor.celsius = temperatura;

    resultado.innerHTML = '
      Fahrenheit: ${conversor.celsiusToFahrenheit().toFixed(2)} grados F<br>
      Kelvin: ${conversor.celsiusToKelvin().toFixed(2)} K
    ';
  }

```

Paso 18: Convertir desde Fahrenheit

Si unidad es fahrenheit, se guarda la temperatura en conversor.fahrenheit. El llamado a conversor.fahrenheitToKelvin() esta correcto en tu codigo actual.

```

if (unidad === 'fahrenheit') {
  conversor.fahrenheit = temperatura;

  resultado.innerHTML = '
    Celsius: ${conversor.fahrenheitToCelsius().toFixed(2)} grados C<br>
    Kelvin: ${conversor.fahrenheitToKelvin().toFixed(2)} K
  ';
}

```

Paso 19: Convertir desde Kelvin

Si unidad es kelvin, se guarda la temperatura en conversor.kelvin y se calculan Celsius y Fahrenheit.

```

if (unidad === 'kelvin') {
  conversor.kelvin = temperatura;

  resultado.innerHTML = '
    Celsius: ${conversor.kelvinToCelsius().toFixed(2)} grados C<br>
    Fahrenheit: ${conversor.kelvinToFahrenheit().toFixed(2)} grados F
  ';
}

```

Revision final del codigo actual

- Bien: import/export estan alineados con script type="module".
- Bien: los IDs usados en script.js existen en index.html.
- Bien: la variable conversor esta escrita correctamente en las conversiones.
- Bien: los labels for ya coinciden con sus inputs/select.
- Bien: no queda texto suelto antes del script type="module" en index.html.
- No pendiente: no cambiar los textos con tilde solo porque esta terminal los muestre raro.

Idea central

funciones.js se encarga de calcular. script.js se encarga de hablar con la pagina: leer el input, validar, llamar los metodos y escribir el resultado.